



WARWICK

Advanced Computer Graphics

Light, HDR, and Film

Dr. Tom Bashford-Rogers

Overview

- ▶ Lighting and Displays
- ▶ High Dynamic Range

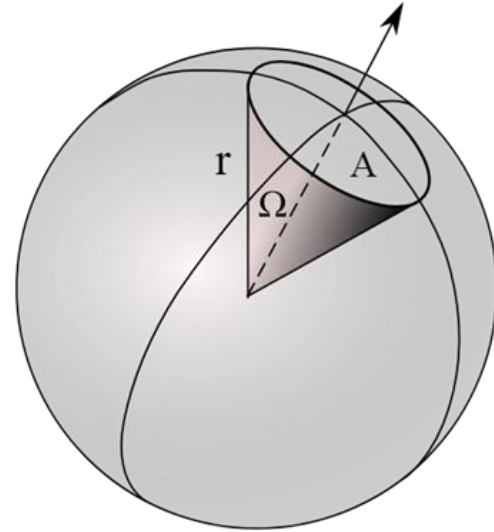


Background



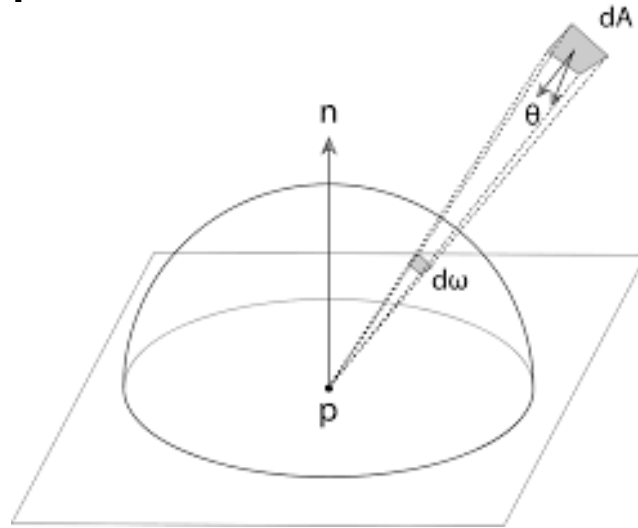
Quantities

- Wavelength λ : m
- Area A : m^2
- Angle θ : *radians*
- Solid Angle Ω : *sr*



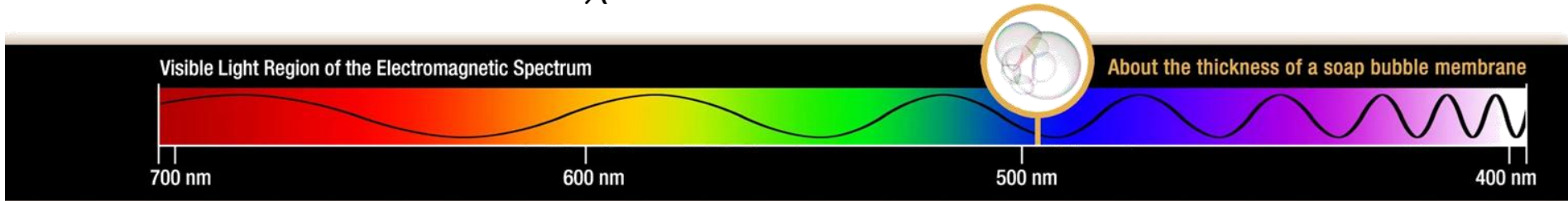
Background

- Differential Solid Angle: $d\omega$
 - Area on a sphere “around” a direction (x, y, z)



Light

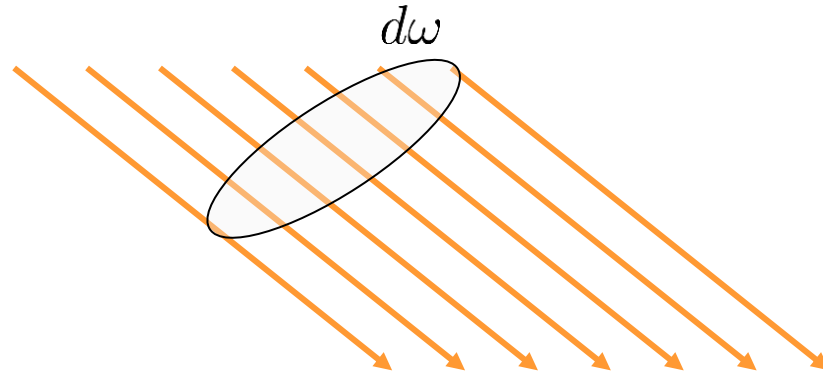
► Energy $\mathbb{E} = \frac{hc}{\lambda}$



► Power (radiant flux) $\Phi = \frac{d\mathbb{E}}{dt}$

Light

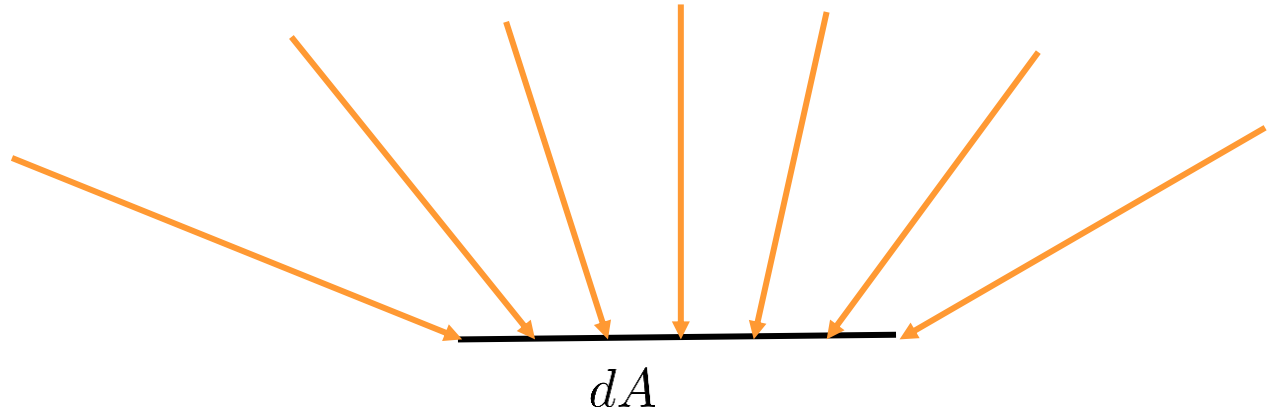
- ▶ “Particles” (photons) of light flowing through space: Intensity $\mathbb{I} = \frac{d\Phi}{d\omega}$



- Coming from a differential solid angle

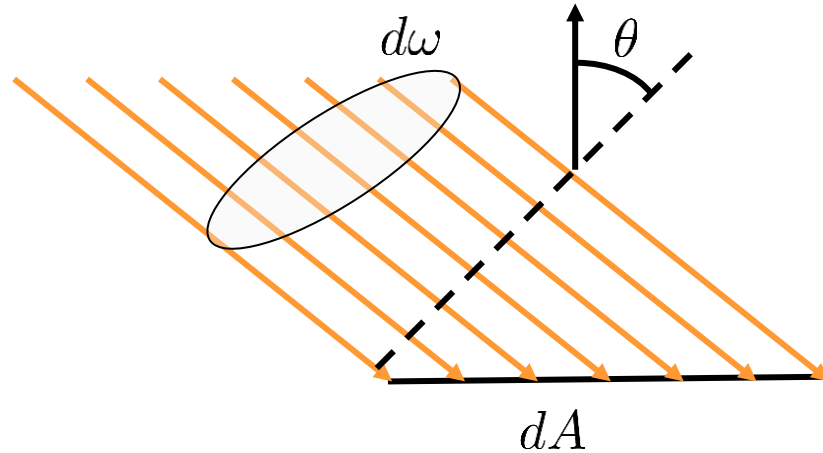
Light

- ▶ These can hit a differential surface of size dA
 - Irradiance $E = \frac{d\Phi}{dA}$



Light

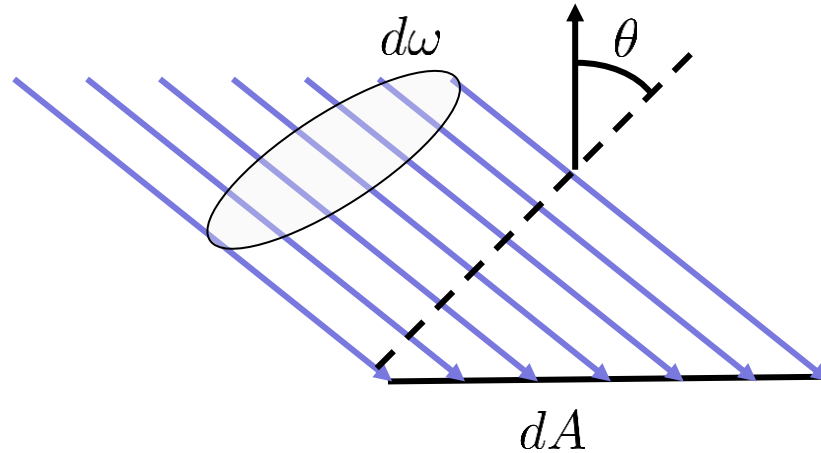
- These can hit a differential surface of size dA at an angle θ : Radiance $L = \frac{d^2\Phi}{dA \cos \theta d\omega}$



Light

► At different wavelengths

– Spectral Radiance $L(\lambda) = \frac{d^2\Phi}{dA \cos\theta d\lambda d\omega}$



Light

► Radiance

$$L = \frac{d^2\Phi}{dA \cos \theta d\omega}$$

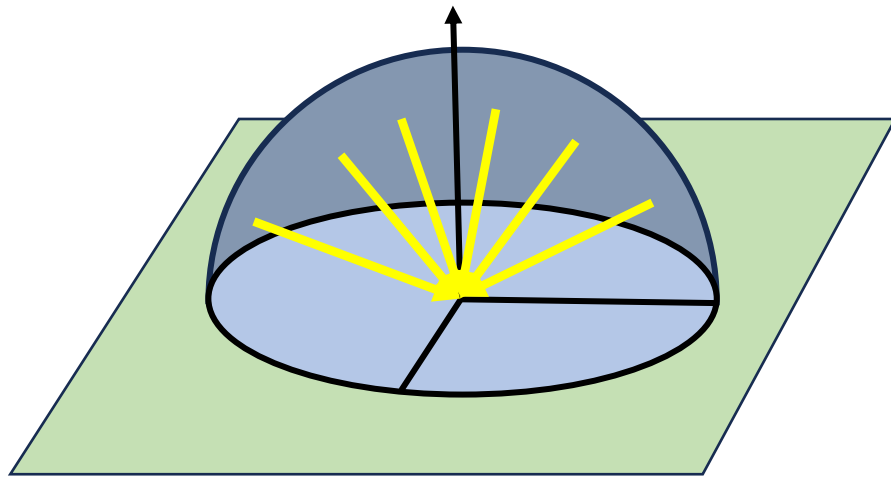
► Most important quantity in graphics!



Light

- ▶ Link between Radiance and Irradiance
 - Light coming to a point from all directions

$$E = \int_{\Omega} L \cos(\theta) d\theta$$

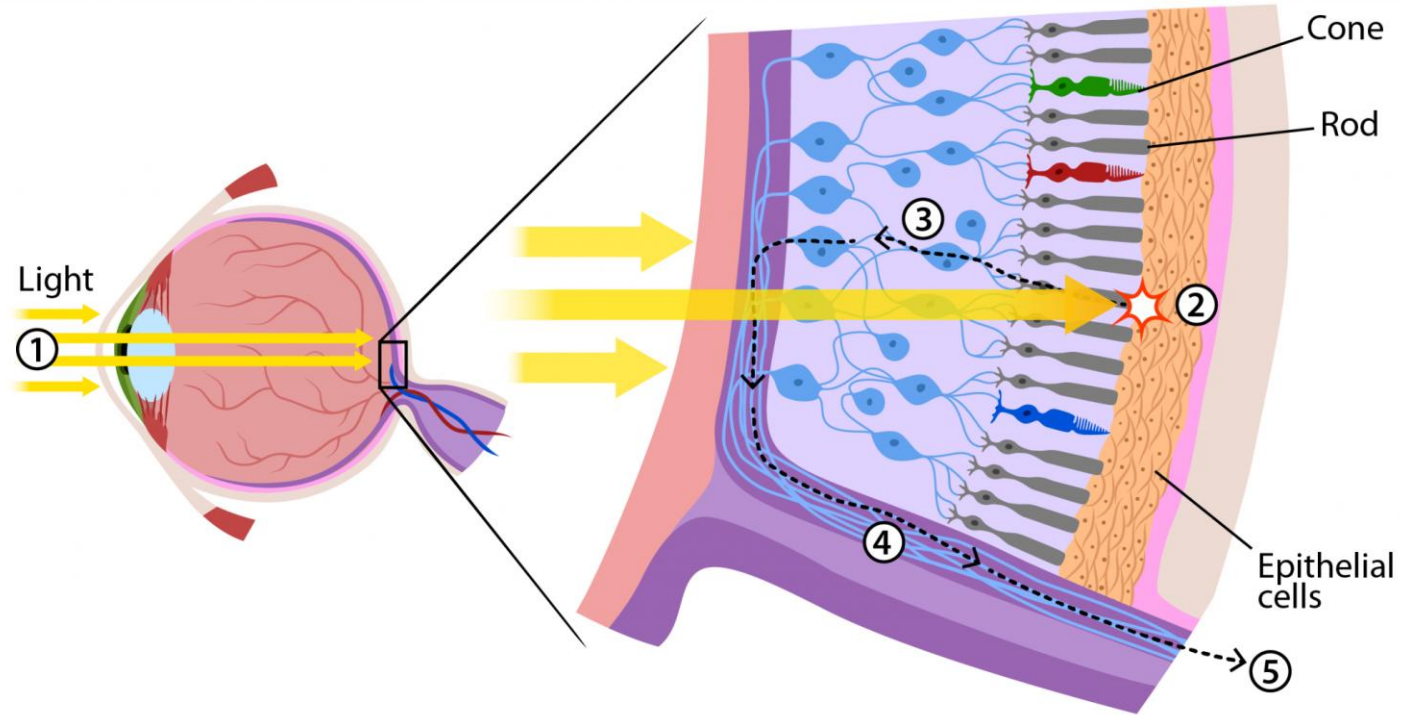


Colour Science

- ▶ How we perceive colours and how they are represented mathematically and physically
- ▶ Eye's Receptors
 - Cones in the retina (L, M, S types for long, medium, short wavelengths)



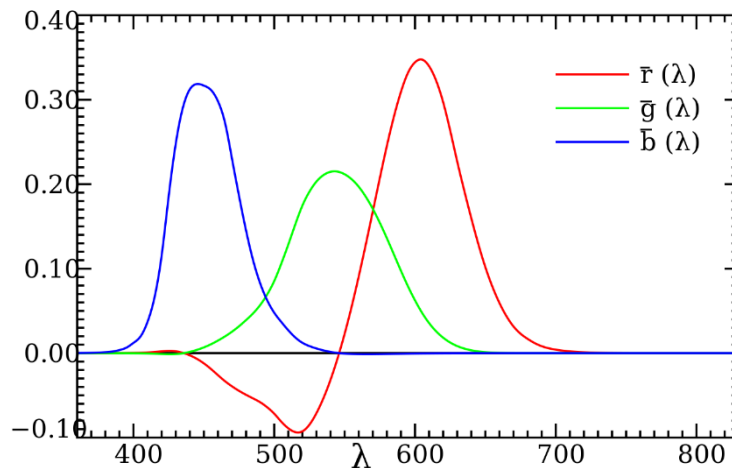
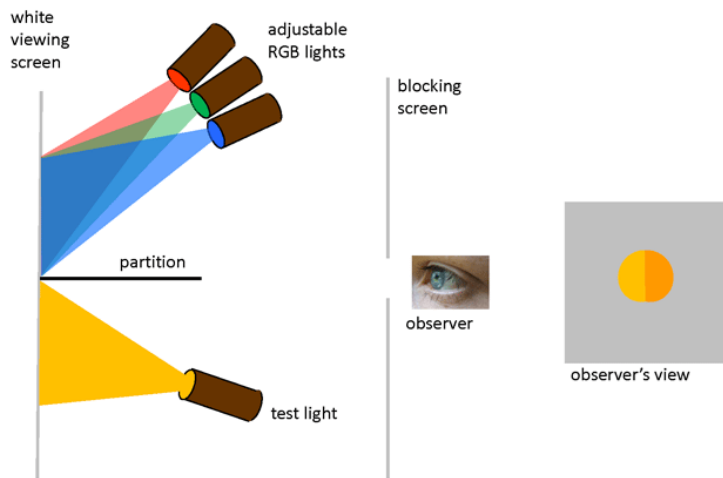
Colour Science



Colour Science

► Tristimulus Theory

– CIE 1931 RGB color matching functions

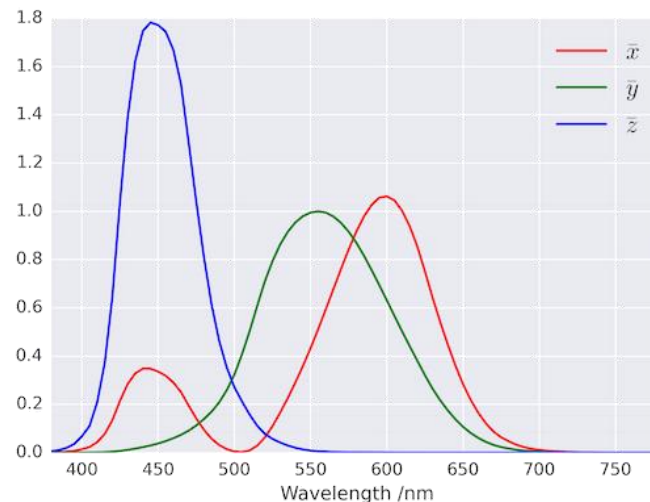


Colour Science

► Tristimulus Theory

– CIE colour matching functions $\bar{x}(\lambda)$, $\bar{y}(\lambda)$, $\bar{z}(\lambda)$

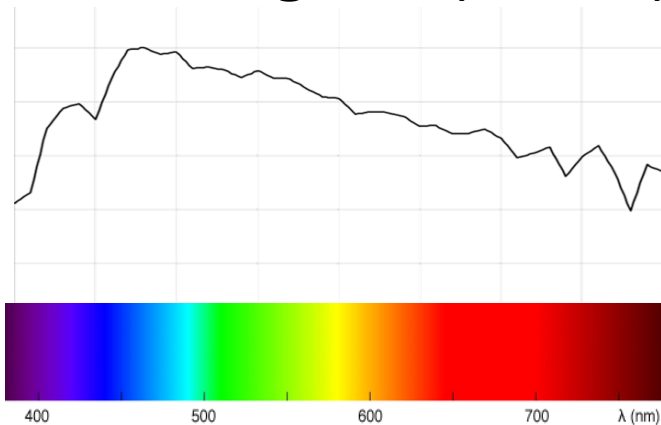
$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4125 & 0.3576 & 0.1804 \\ 0.2127 & 0.7152 & 0.0722 \\ 0.0193 & 0.1192 & 0.9503 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$



Colour Science

► Tristimulus Theory

- Colour is represented as a combination of three signals (X, Y, Z)



$$X = \int_{\lambda} L(\lambda) \bar{x}(\lambda) d\lambda$$

$$Y = \int_{\lambda} L(\lambda) \bar{y}(\lambda) d\lambda$$

$$Z = \int_{\lambda} L(\lambda) \bar{z}(\lambda) d\lambda$$

Colour Science

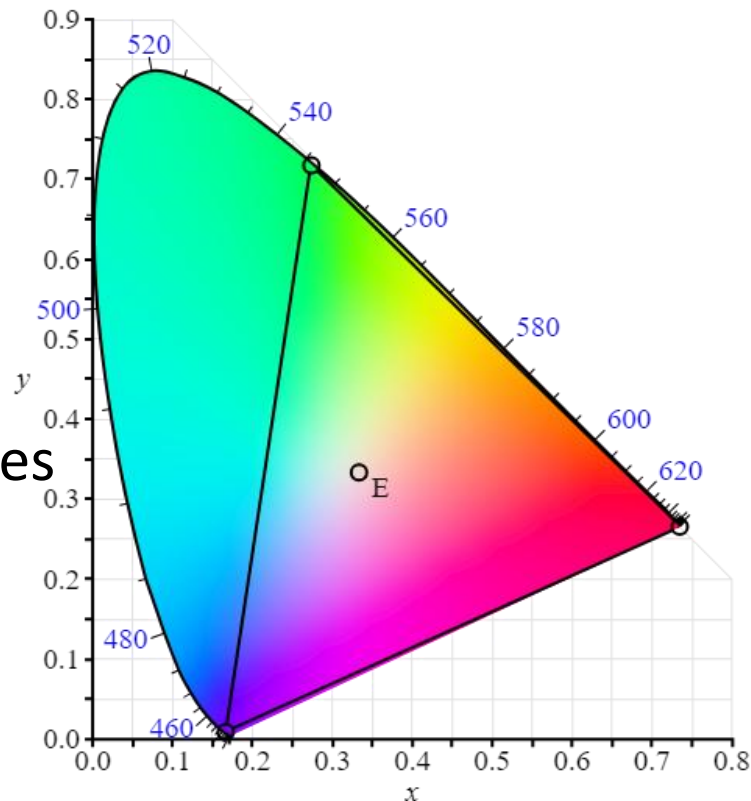
► Chromaticity Diagram

– Visual representation

- Based on CIE XYZ values
- Mapped as x and y coordinates

$$x = \frac{X}{X + Y + Z}$$

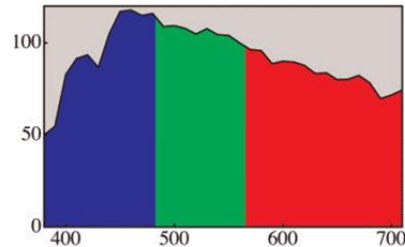
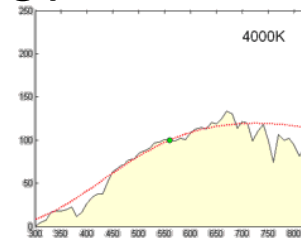
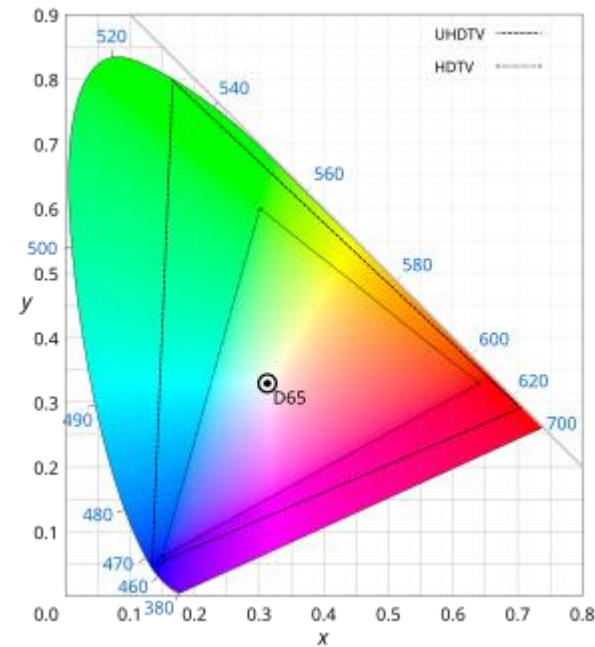
$$y = \frac{Y}{X + Y + Z}$$



https://en.wikipedia.org/wiki/CIE_1931_color_space

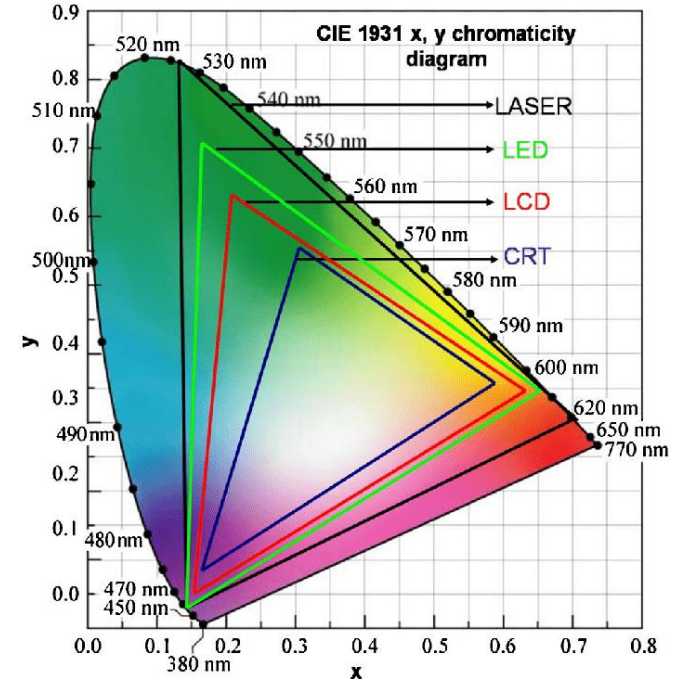
Colour Science

- ▶ Chromaticity Diagram
 - The three points on the diagram correspond to primaries
 - Whitepoint chosen to represent white given the chromacity of the illuminant
 - Common to use D65



Colour Science

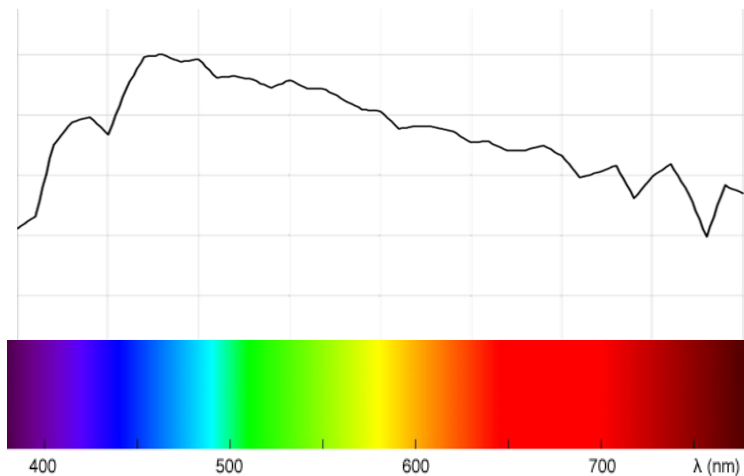
- ▶ Display colours
 - Subset of colour space
 - Different displays



Colour Science

► Colour Spaces: Linear sRGB

- Can convert to colour space from XYZ



$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 3.2405 & -1.5371 & -0.4985 \\ -0.9693 & 1.8760 & 0.0416 \\ 0.0556 & -0.2040 & 1.0572 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

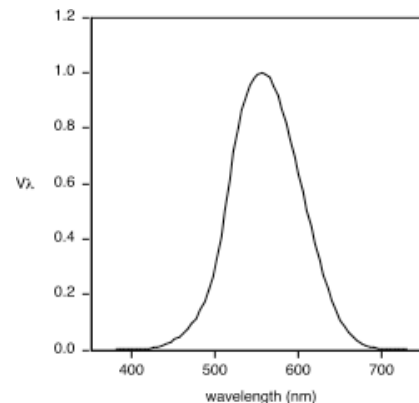


Colour Science

► Luminance

- From Y component of XYZ
- Corresponds closely to the human eye's brightness sensitivity

$$Y = 0.2126 R + 0.7152 G + 0.0722 B$$



Colour Science

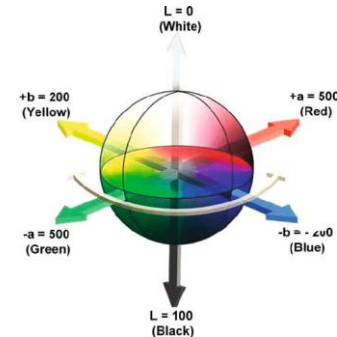
► Colour Spaces: Lab

– Differences in coordinates relate to perceptual differences

– L (Lightness),

– a^* (green-red)

– b^* (blue-yellow)



$$L^* = 116 \left(\frac{Y}{Y_n} \right)^{1/3} - 16$$

$$a^* = 500 \left(\left(\frac{X}{X_n} \right)^{1/3} - \left(\frac{Y}{Y_n} \right)^{1/3} \right)$$

$$b^* = 200 \left(\left(\frac{Y}{Y_n} \right)^{1/3} - \left(\frac{Z}{Z_n} \right)^{1/3} \right)$$

Evaluation of the role of image resolution in the perception of color differences

Colour Science



► Colour Spaces: CMYK

- Cyan, Magenta, Yellow, Key/Black
- Used in colour printing
 - Colours are created by subtracting light

$$C' = 1 - R$$

$$M' = 1 - G$$

$$Y' = 1 - B$$

$$K = 1 - \max(C', M', Y')$$

$$C = \frac{C' - K}{1 - K}, \quad M = \frac{M' - K}{1 - K}, \quad Y = \frac{Y' - K}{1 - K}$$



Colour Science

► Colour Spaces:

- HSL / HSV
 - Image editing
- Luv
 - Colour difference calculations
- YCbCr
 - Image and video compression



Displays

▶ CRT (Cathode Ray Tube)

- Electron beams to illuminate phosphor on a glass screen



▶ LCD (Liquid Crystal Display)

- Liquid crystals between polarised glass layers.



▶ (O/Q)LED (Light Emitting Diode)

- LED backlighting for LCD display, +brightness and contrast



Gamma Correction and SRGB

▶ CRT has non-linear luminance values

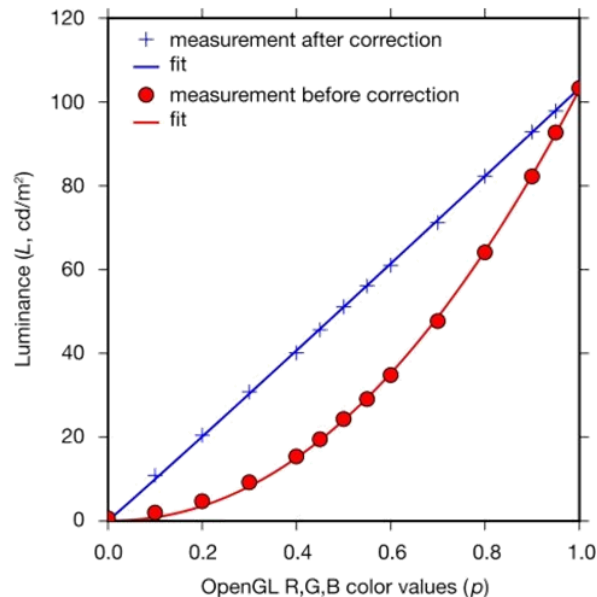
– LCD / LED have followed

▶ Gamma curve to correct

$$L_{out} = L_{in}^{\frac{1}{\gamma}}$$

– Dependent on display

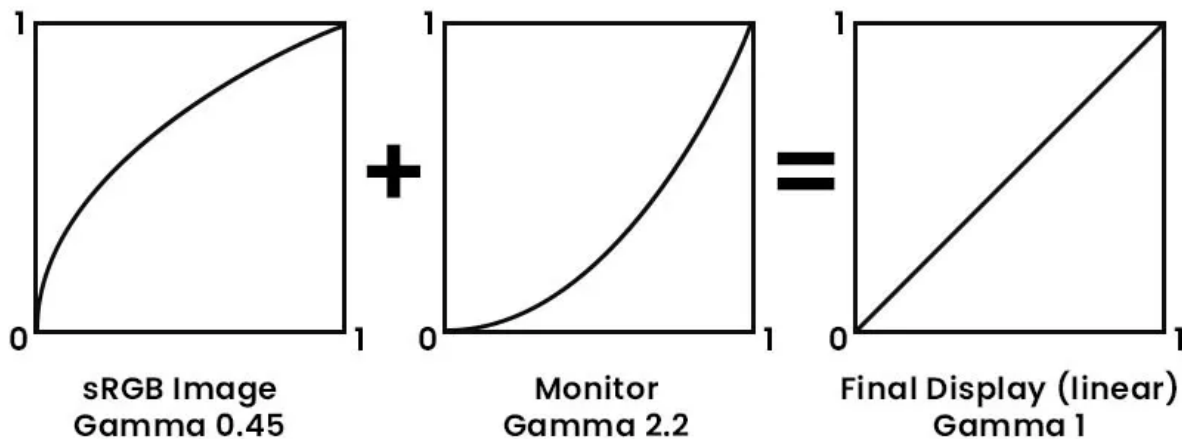
– Usually $\gamma = 2.2$



Vision Egg: An Open-Source Library for Realtime Visual Stimulus Generation

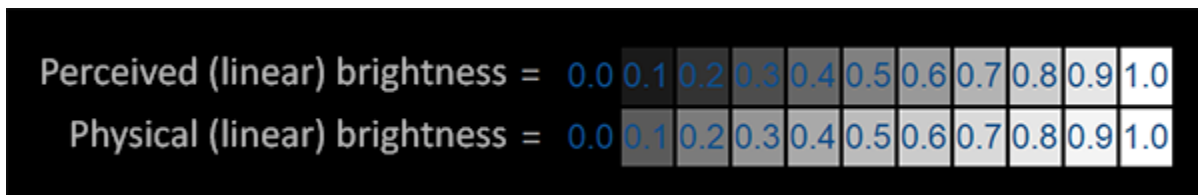
Gamma Correction and SRGB

► Correct to linear



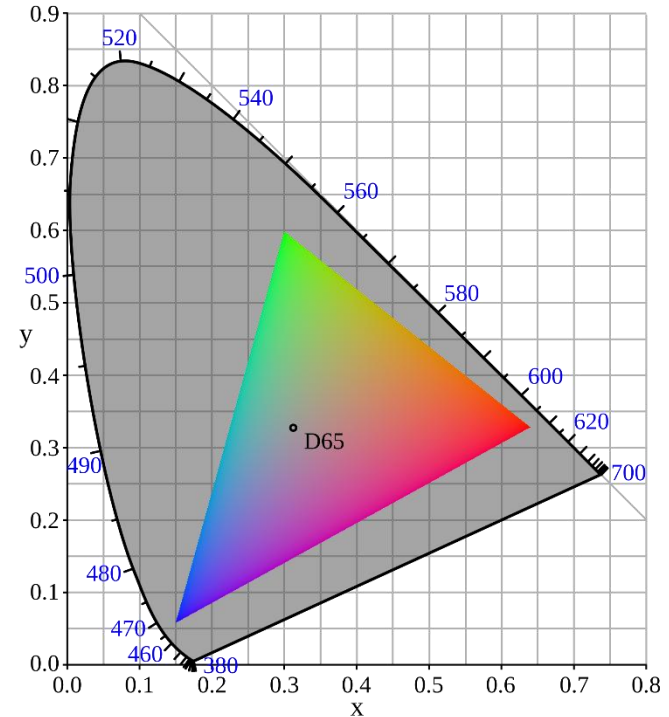
Gamma Correction and SRGB

- ▶ sRGB defines a standard colour space across devices
 - Linear response at low intensities, 2.4 gamma for higher intensities (approximates 2.2)



Gamma Correction and SRGB

- ▶ Cannot represent all colours
- ▶ Chosen to represent colours displayable by screens



Gamma Correction and sRGB

- We usually want to work with linear (*lin*) colours $C \in \{R, G, B\}$, need to convert between spaces

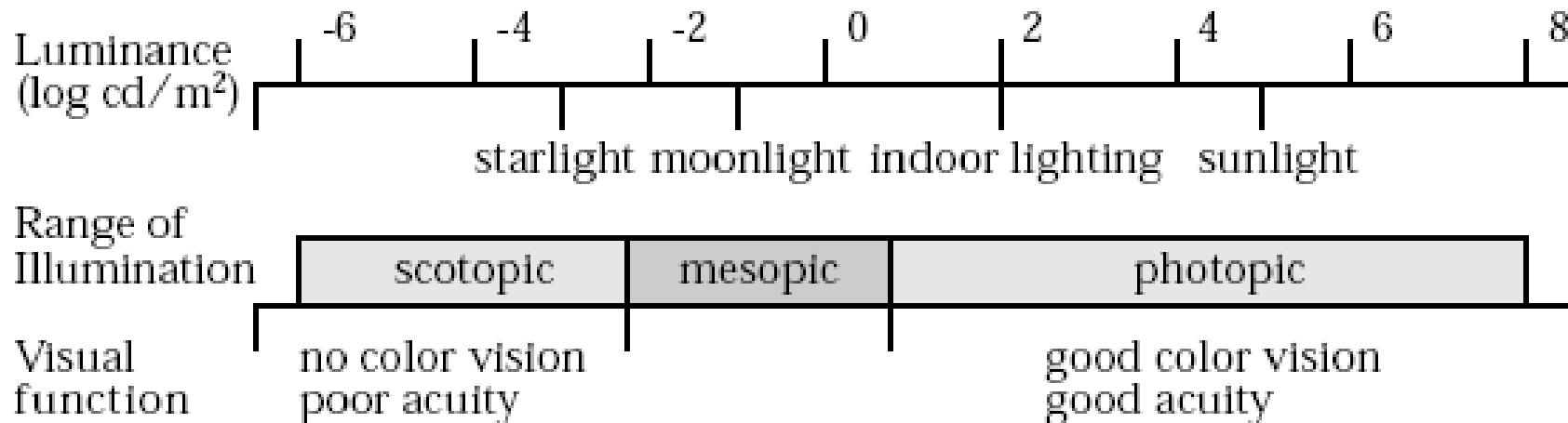
Gamma		sRGB
$C_{lin} = C_{gamma}^\gamma$	$C_{lin} = \begin{cases} \frac{C_{sRGB}}{12.92} & \text{if } C_{sRGB} \leq 0.04045 \\ \left(\frac{C_{sRGB} + 0.055}{1.055}\right)^{2.4} & \text{if } C_{sRGB} > 0.04045 \end{cases}$	
$C_{gamma} = C_{lin}^{\frac{1}{\gamma}}$	$C_{sRGB} = \begin{cases} 12.92 \cdot C_{lin} & \text{if } C_{lin} \leq 0.0031308 \\ 1.055 \cdot C_{lin}^{\frac{1}{2.4}} - 0.055 & \text{if } C_{lin} > 0.0031308 \end{cases}$	

High Dynamic Range Images

- ▶ Definitions
- ▶ Tone Mapping
- ▶ Inverse Tone Mapping

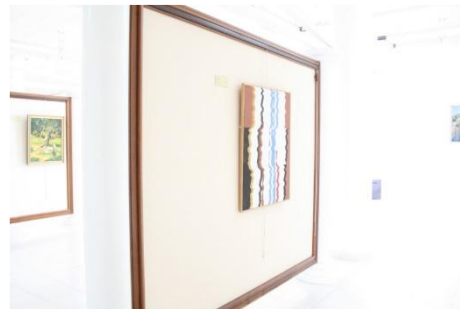
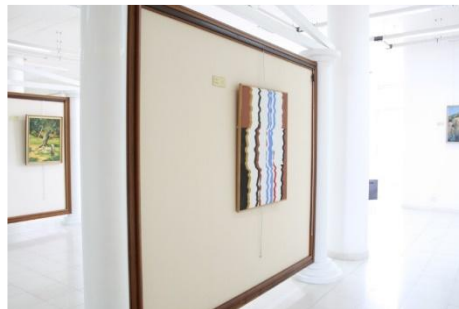
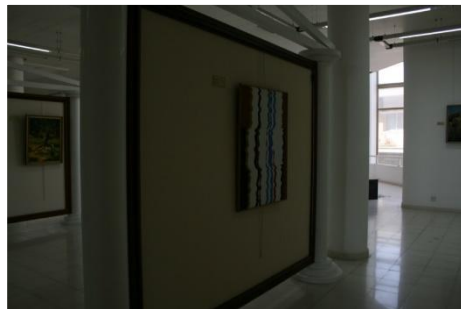
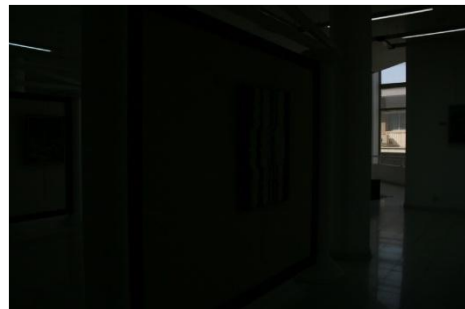


High Dynamic Range Images



High Dynamic Range Images

- ▶ Stores real-world lighting values



High Dynamic Range Images

► HDR Pipeline

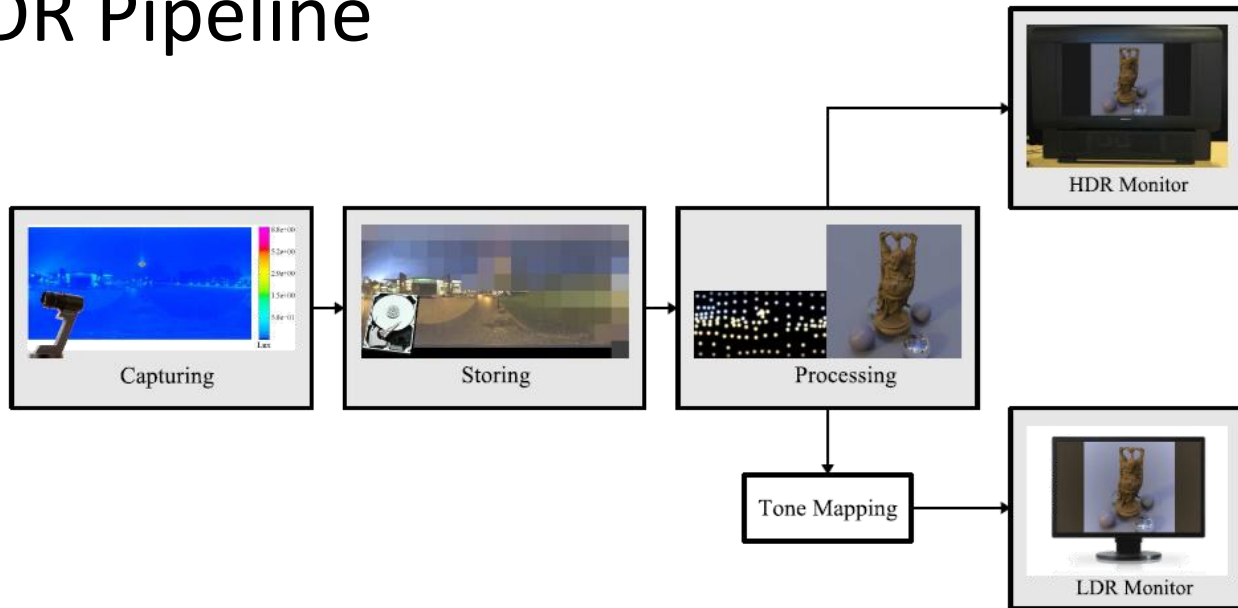


Image courtesy of Banterle et al.

High Dynamic Range Images

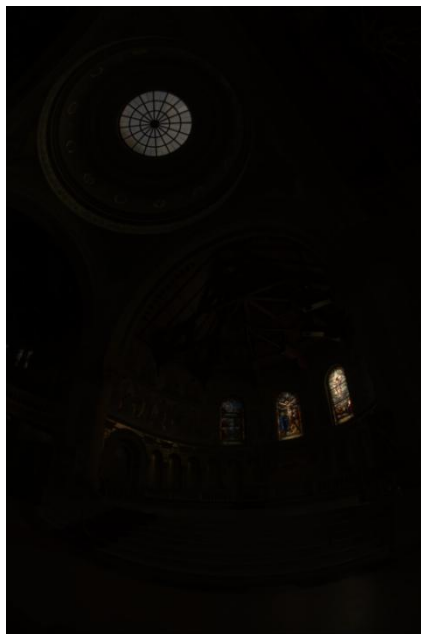
► Creation

- Capture using photography
 - In certain cases: standard cameras
 - Use specialised cameras
- Create by rendering
 - Generate physically-based images
- Expanding from legacy LDR content



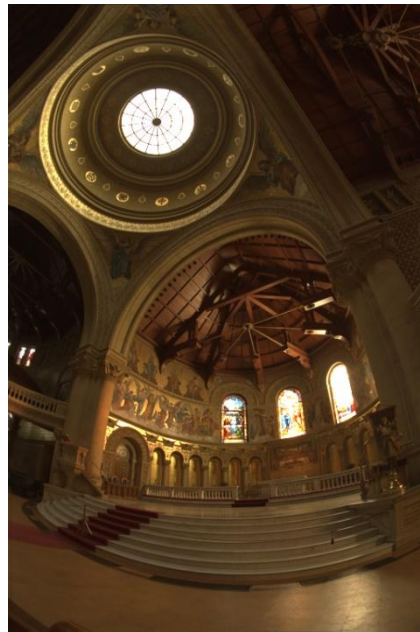
High Dynamic Range Images

► Creation: Standard cameras



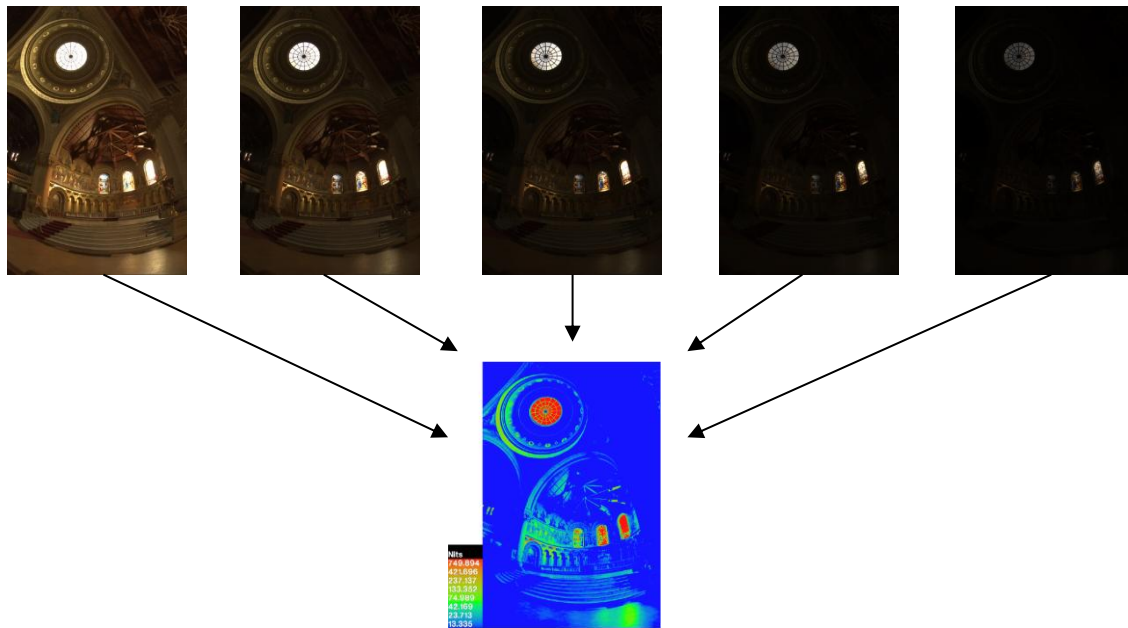
Scenes with
HDR

Single
exposure
insufficient



High Dynamic Range Images

► Creation: Standard cameras



High Dynamic Range Images

► Creation: Standard cameras

– Merging (Debevec)

- Stack of P images Z_{ij} (pixel i , image j) at exposure times Δt_j
- Camera response function $f(Z_{ij})$
- Weighting function to filter outliers $w(Z_{ij})$

$$\ln(E_i) = \frac{\sum_{j=1}^P w(Z_{ij}) (f(Z_{ij}) - \ln(\Delta t_j))}{\sum_{j=1}^P w(Z_{ij})}$$

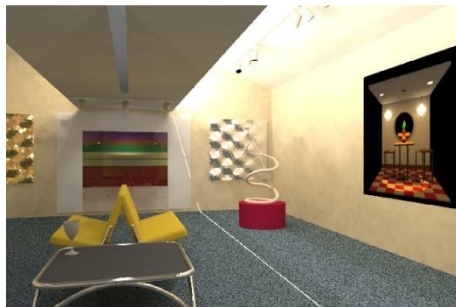
High Dynamic Range Images

- ▶ Creation: Use specialised cameras



High Dynamic Range Images

► Creation: Rendering



High Dynamic Range Images

► Storing

- RAW (pfm): 32 bits per colour channel
 - HUGE: 24Mb for HD HDR image
- EXR: 16 Bit floating point, compressed
- HDR: 4 bytes per pixel, RGB + Shared exponent
- Tiff: Flexible, supports many storage methods



High Dynamic Range

► Colour class

► Constructors

```
Colour() {}  
Colour(float _r, float _g, float _b) {  
    r = _r;  
    g = _g;  
    b = _b;  
}  
Colour(unsigned char _r, unsigned char _g, unsigned char _b, unsigned char _a)  
{  
    r = (float)_r / 256.0f;  
    g = (float)_g / 256.0f;  
    b = (float)_b / 256.0f;  
}
```

```
class Colour  
{  
public:  
    float r;  
    float g;  
    float b;
```

High Dynamic Range

► Useful operators

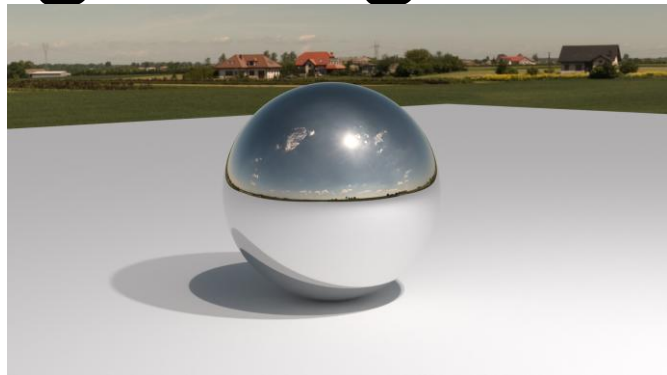
```
Colour operator+(const Colour& colour) const  
Colour operator*(const Colour& colour) const  
Colour operator*(const float v) const  
Colour operator/(const float v) const
```

► Method to calculate luminance

$$Y = 0.2126 R + 0.7152 G + 0.0722 B$$

High Dynamic Range Images

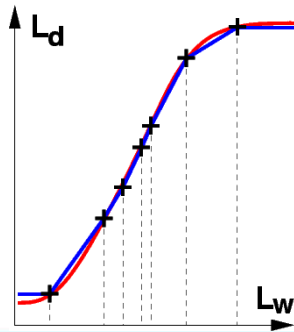
- ▶ Processing
 - Compression
 - Rendering



High Dynamic Range Images

► Display

- Present “directly” on HDR displays
 - Still needs to be mapped
- Map HDR values to LDR for conventional displays



High Dynamic Range Images

► Display

- Tone Mapping: $M : \mathbb{R}^+ \mapsto [0..255]$ $L_{\text{out}} = M(L_{\text{in}})$
- Map luminance then display: $L_{\text{disp}} = \text{clamp}(L_{\text{out}}, 0, 1) \cdot 255$
- Global operators
 - Tone map each pixel with same operator across the image
- Local operators
 - Tone mapping for each pixel depends on nearby pixels

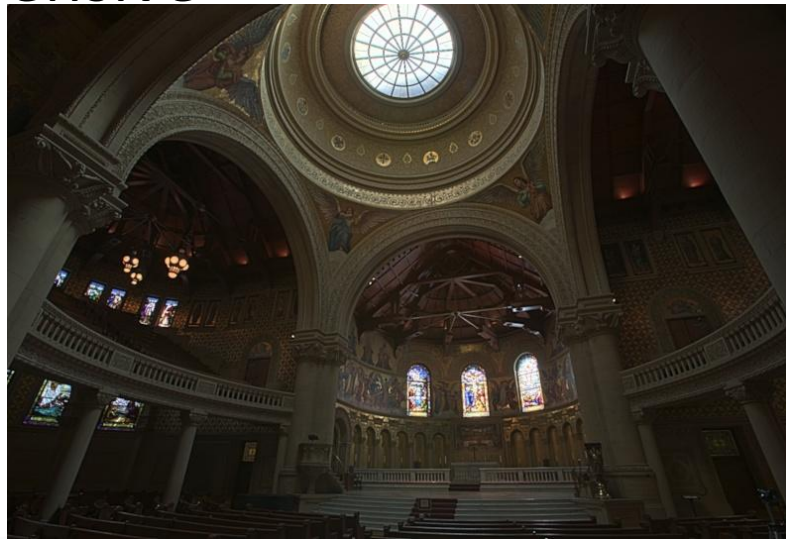
High Dynamic Range Images

- ▶ Global operators
 - Computationally cheap
 - May miss some details



High Dynamic Range Images

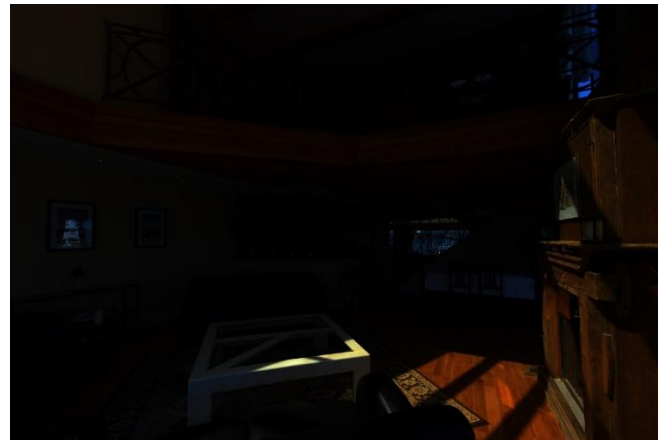
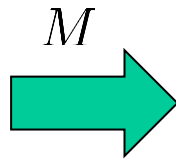
- ▶ Local operators
 - Computationally more expensive
 - Not often used in games
 - More details



High Dynamic Range Images

► Tone mapping operators in games

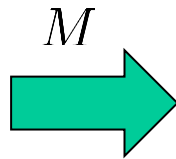
- Linear without gamma $L_{\text{out}} = \frac{L_{\text{in}}}{L_{\text{max}}}$



High Dynamic Range Images

► Tone mapping operators in games

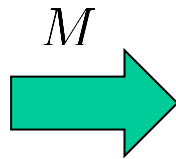
- Linear with gamma $L_{\text{out}} = \left(\frac{L_{\text{in}}}{L_{\text{max}}} \right)^{\frac{1}{2.2}}$



High Dynamic Range Images

► Tone mapping operators in games

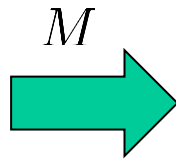
- Linear with gamma and exposure $L_{\text{out}} = (L_{\text{in}} * 2^e)^{\frac{1}{2.2}}$



High Dynamic Range Images

► Tone mapping operators in games

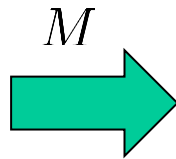
- Reinhard Global $L_{\text{out}} = \left(\frac{L_{\text{in}}}{1 + L_{\text{in}}} \right)^{\frac{1}{2.2}}$



High Dynamic Range Images

► Tone mapping operators in games

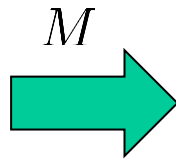
- Filmic (Uncharted, Hable) $L_{\text{out}} = \left(\frac{C(L_{\text{in}})}{C(W)} \right)^{\frac{1}{2.2}}$



High Dynamic Range Images

► Tone mapping operators in games

– Filmic:
$$C(x) = \frac{(x \cdot (A \cdot x + C \cdot B) + D \cdot E)}{(x \cdot (A \cdot x + B) + D \cdot F)} - \frac{E}{F}$$



High Dynamic Range Images

► Tone mapping operators in games

– Filmic Parameters

$$A = 0.15$$

$$B = 0.50$$

$$C = 0.10$$

$$D = 0.20$$

$$E = 0.02$$

$$F = 0.30$$

$$W = 11.2$$



High Dynamic Range Images

► Creation: Inverse Tone Mapping

- Map from LDR to HDR $M^{-1} : [0..255] \mapsto \mathbb{R}^+$

Inverse Tone Mapping

Francesco Banterle ^{*}
University of Bristol

Patrick Ledda [†]
University of Bristol

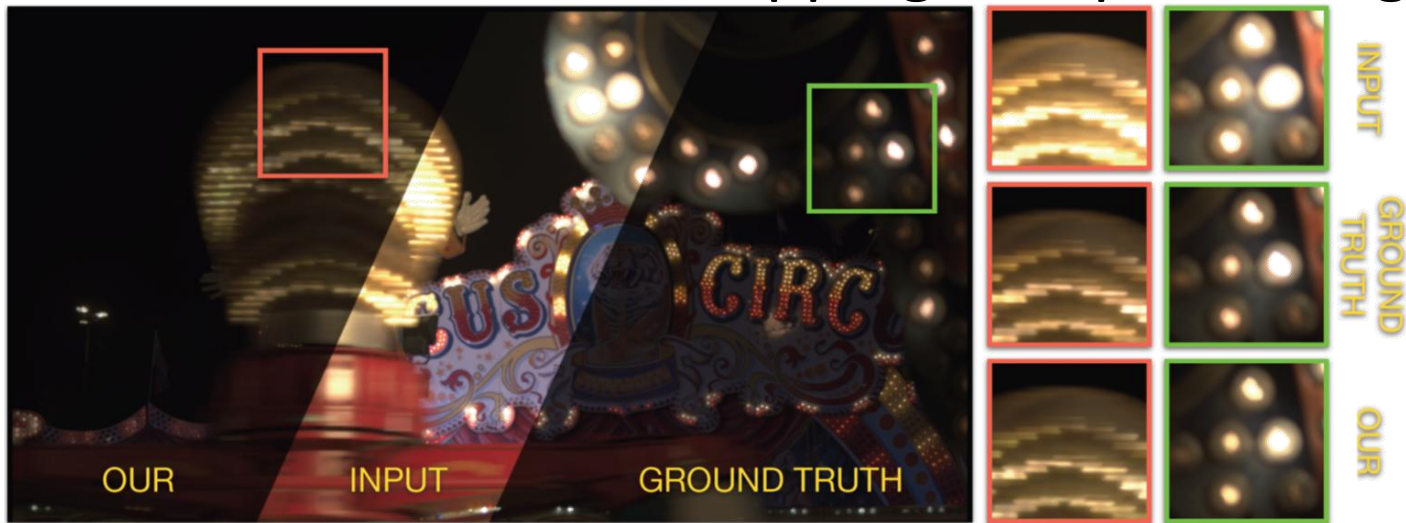
Kurt Debattista ^{*}
University of Bristol

Alan Chalmers [§]
University of Bristol

- Invert the tone mapping equations
 - Guess luminance values

High Dynamic Range Images

- ▶ Creation: Inverse Tone Mapping
 - Modern inverse tone mapping: Deep Learning



Rendering with HDR

- ▶ Implementation
- ▶ Need to write and tonemap HDR values
- ▶ Film class in Imaging.h



Film Implementation

► Film Class

```
class Film
{
public:
    Colour* film;
    int width;
    int height;
    int SPP;
    ImageFilter* filter;
```

Film Implementation

► Film Class

– Useful functions

```
void clear()
{
    memset(film, 0, width * height * sizeof(Colour));
    SPP = 0;
}
void incrementSPP()
{
    SPP++;
}
```

Film Implementation

► Tonemapping

- Map a pixel from film to unsigned char
- Something like:

```
void tonemap(int x, int y, unsigned char& r, unsigned char& g, unsigned char& b)
```

- Implement one of the methods from earlier

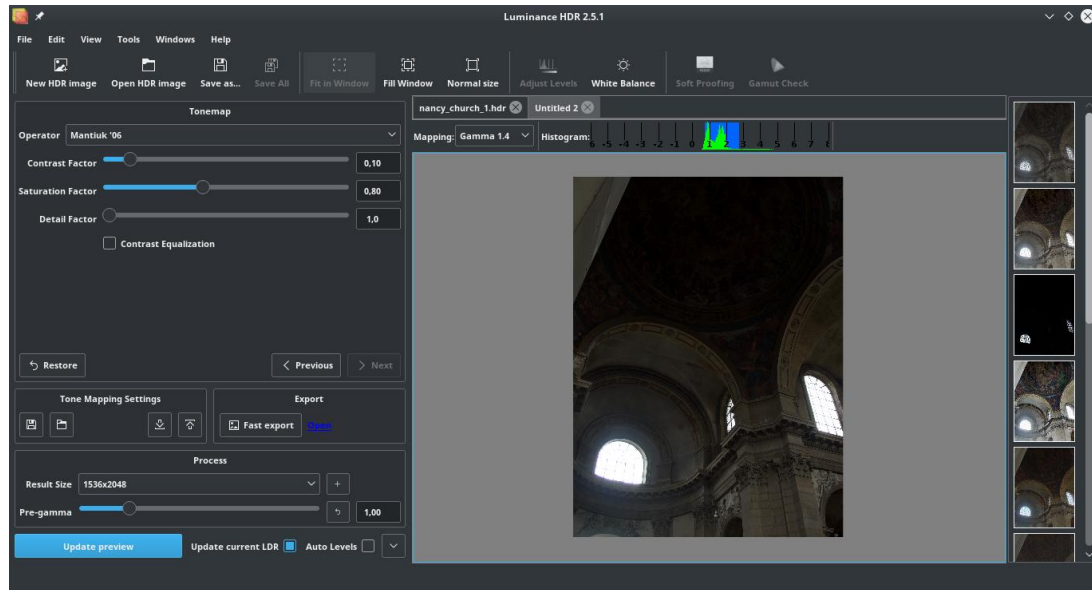


Software to view HDR Files

▶ LumianceHDR

▶ Plenty of others

- Find one you like



Film

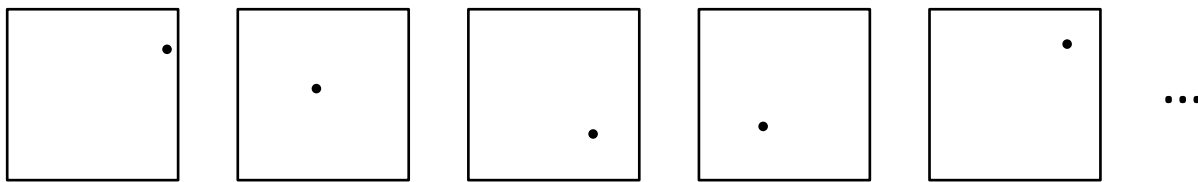
- ▶ Need to write and accumulate HDR values
- ▶ Emulates a camera film
 - Each pixel: $I(x, y)$



Film

► Accumulating HDR values

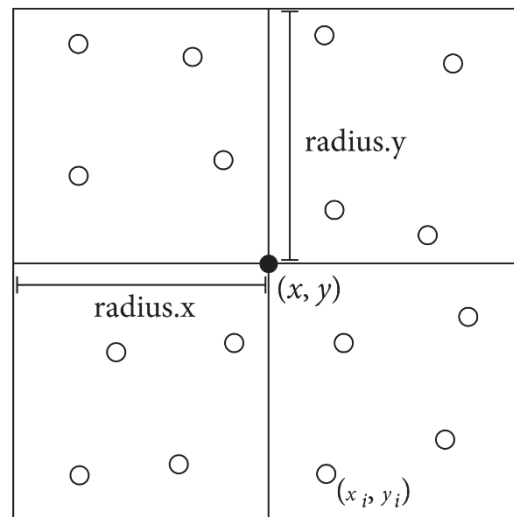
- Will create samples $(x, y, L(x, y))$
- We will be accumulating samples to the film



- Options to do this
 - Filtering

Filtering

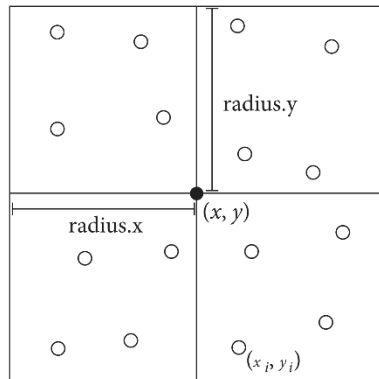
- ▶ Accumulating HDR values
- ▶ Filtering
 - Control over contributions
 - Can improve quality
 - Based on a filter kernel



Filtering

► Filter kernel

- Contribute to pixels within radius
- Define kernel as a function of a distance from sample point to pixel i



$$h(x - x_i, y - y_i) = h(\Delta x, \Delta y)$$

$$\Delta x = x - x_i$$

$$\Delta y = y - y_i$$

Filtering

► Filter kernel

- Contribute to pixels within radius
- Must conserve energy

$$C(x_i, y_i) = \frac{h(x - x_i, y - y_i)L(x, y)}{\sum_{j=1}^{radius} h(x - x_j, y - y_j)}$$

- Pixel value accumulates contribution

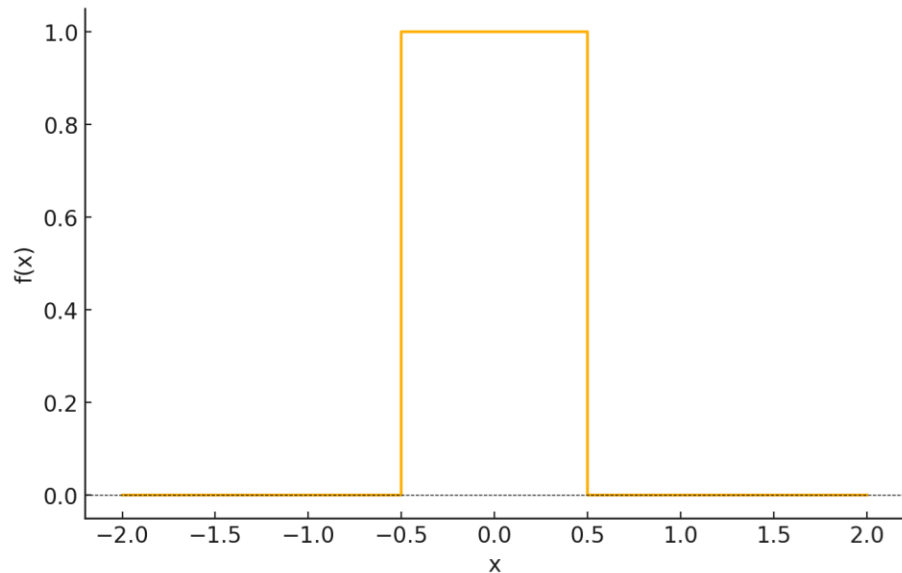


Filtering

► Filter kernels: Box

- Simplest
- radius = 1
- Filter value:

$$h(\Delta x, \Delta y) = 1$$

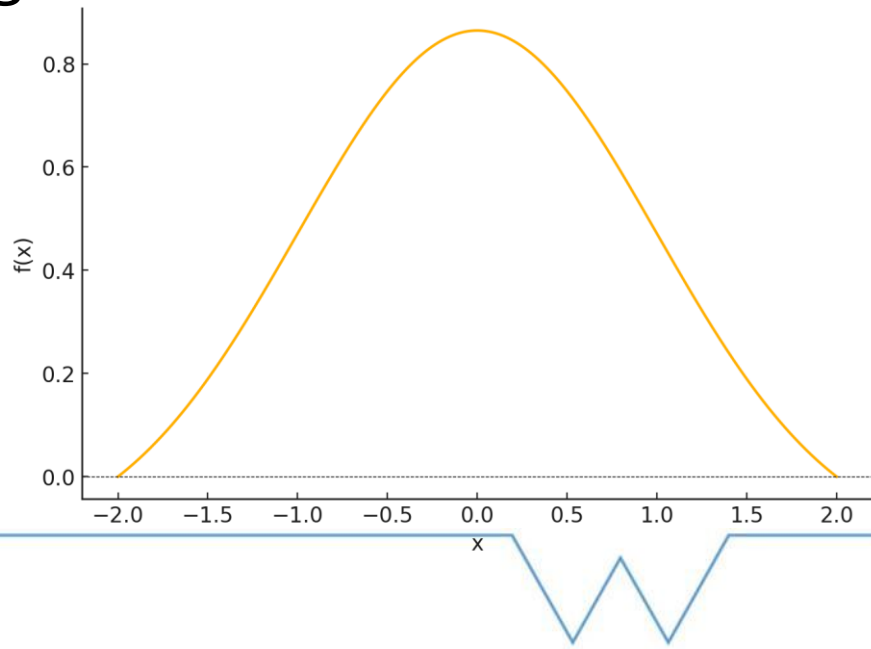


Filtering

► Filter kernels: Gaussian

- Define Gaussian which goes to 0 at radius:

$$G(d, radius) = e^{-\alpha d^2} - e^{-\alpha radius^2}$$



Filtering

► Filter kernels: Gaussian

- Filter value

$$h(\Delta x, \Delta y) = G(\Delta x, radius) \cdot G(\Delta y, radius)$$

- Gaussian is separable
- Can choose any radius
 - Need to balance smoothness and detail

Filtering

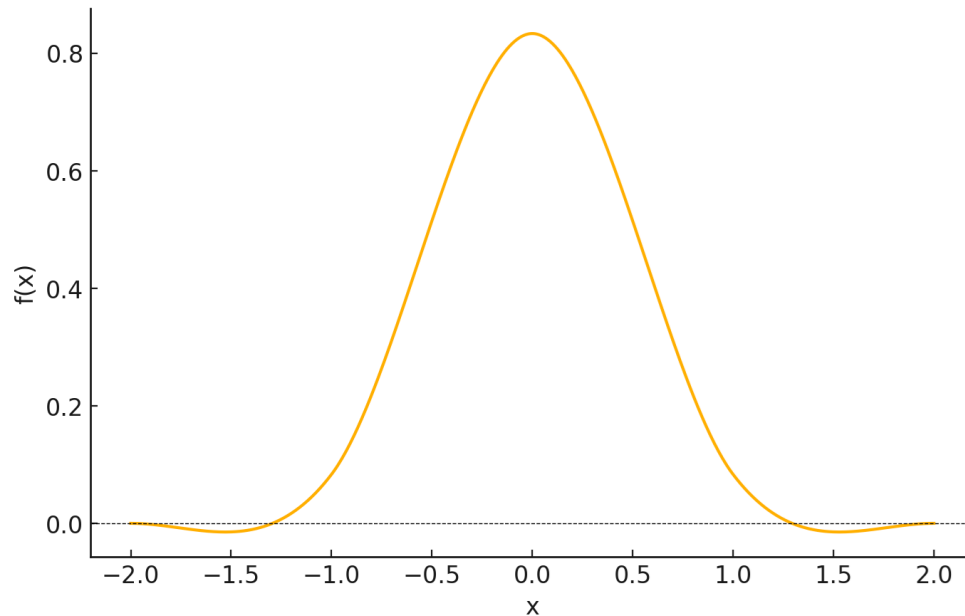
► Filter kernels: Mitchell-Netravali

- Filter value
- B and C influence shape
- Usually $B = \frac{1}{3}$
 $C = \frac{1}{3}$
- Radius fixed at 2

$$h_m(x) = \begin{cases} \frac{1}{6} \left[(12 - 9B - 6C) |x|^3 \right. \\ \quad \left. + (-18 + 12B + 6C) |x|^2 \right. \\ \quad \left. + (6 - 2B) \right], & \text{for } 0 \leq |x| < 1, \\ \frac{1}{6} \left[(-B - 6C) |x|^3 \right. \\ \quad \left. + (6B + 30C) |x|^2 \right. \\ \quad \left. + (-12B - 48C) |x| \right. \\ \quad \left. + (8B + 24C) \right], & \text{for } 1 \leq |x| < 2, \\ 0, & \text{for } |x| \geq 2. \end{cases}$$

Filtering

► Filter kernels: Mitchell-Netraval



Filtering

► Filter kernels: Mitchell-Netravali

- Filter value

$$h(\Delta x, \Delta y) = h_m(\Delta x, radius) \cdot h_m(\Delta y, radius)$$

- Mitchell-Netravali is also separable



Filtering

► Box



Filtering

► Gaussian



Filtering

- ▶ Mitchell-Netravali



Filter Implementation

► Base class

```
class ImageFilter
{
public:
    virtual float filter(const float dx, const float dy) const = 0;
    virtual int size() const = 0;
};
```

Filter Implementation

► Box Filter

```
class BoxFilter : public ImageFilter {  
public:  
    float filter(float dx, float dy) const {  
        if (fabsf(x) <= 0.5f && fabsf(y) <= 0.5f)  
        {  
            return 1.0f;  
        }  
        return 0;  
    }  
    int size() const  
    {  
        return 0;  
    }  
};
```

Filter Implementation

- ▶ Implement:
 - Gaussian Filter
 - Mitchell-Netravali Filter



Filter Implementation

► Gaussian filter

- Suggest to implement $G(d, radius) = e^{-\alpha d^2} - e^{-\alpha radius^2}$ as a method
- Call from `float filter(float x, float y) const`
- Size related to smoothness



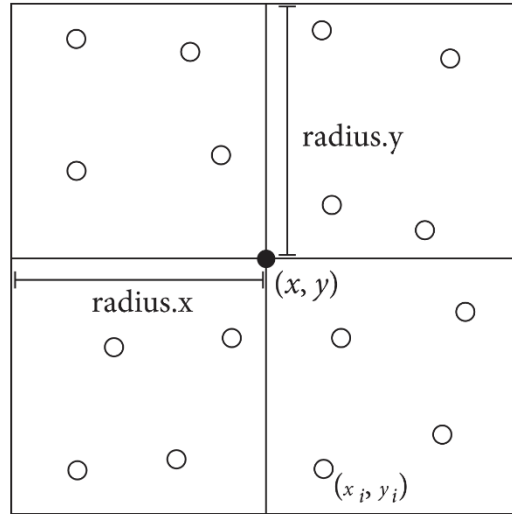
Filter Implementation

- ▶ Mitchell-Netravali filter
 - Implement separable component similar to Gaussian



Filter Implementation

- Need to process pixels



Film Implementation



Splatting samples

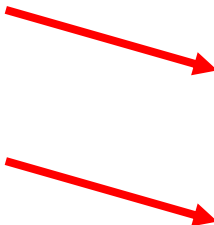
– Example
implementation

```
void splat(const float x, const float y, const Colour &L) {  
    float filterWeights[25]; // Storage to cache weights  
    unsigned int indices[25]; // Store indices to minimize computations  
    unsigned int used = 0;  
    float total = 0;  
    int size = filter->size();  
    for (int i = -size; i <= size; i++) {  
        for (int j = -size; j <= size; j++) {  
            int px = (int)x + j;  
            int py = (int)y + i;  
            if (px >= 0 && px < width && py >= 0 && py < height) {  
                indices[used] = (py * width) + px;  
                filterWeights[used] = filter->filter(px - x, py - y);  
                total += filterWeights[used];  
                used++;  
            }  
        }  
    }  
    for (int i = 0; i < used; i++) {  
        film[indices[i]] = film[indices[i]] + (L * filterWeights[i] / total);  
    }  
}
```

Link to renderer

► Connect film
class to
raytracer

```
void render()
{
    film->incrementSPP();
    for (int y = 0; y < film->height; y++) {
        for (int x = 0; x < film->width; x++) {
            float px = x + 0.5f;
            float py = y + 0.5f;
            Ray ray = scene->camera.generateRay(px, py);
            Colour col = viewNormals(ray);
            //Colour col = albedo(ray);
            film->splat(px, py, col);
            unsigned char r;
            unsigned char g;
            unsigned char b;
            film->tonemap(x, y, r, g, b);
            canvas->draw(x, y, r, g, b);
        }
    }
}
```





Questions?

Computer Graphics

Core Mathematical Concepts



WARWICK

Dr. Tom Bashford-Rogers